Chapter 5: System Implementation, Testing and Evaluation

Intelligent Agent Identity Verification System

5.1 Introduction

This chapter explains how the design approved in Chapter 4 was implemented, the environment in which it ran, and how it was tested and evaluated. The system registers agents, collects identity evidence, combines document and biometric scores, and supports verification decisions and controlled access. It was implemented as a Vue web interface, an Express application server, a Flask artificial intelligence service and a Supabase PostgreSQL database.

The chapter follows a hybrid software and machine learning structure. Sections 5.2 and 5.3 describe the environment, the deviations from the approved design, the implemented modules and workflow, and the dataset and model development procedure. Section 5.4 reports the tests and classification results, Section 5.5 describes the version-control evidence, Section 5.6 declares the AI-assisted tools used, and Section 5.7 discusses the results against the project objectives and states the limitations. The latest saved model, version 3.0.0, achieved 75.800% accuracy on 1,500 fresh synthetic cases. This result was measured under the simulator assumptions described in Section 5.3.6 and does not represent operational identity verification accuracy.

5.2 Implementation Environment

5.2.1 Hardware and Deployment Environment

The system was implemented as a web prototype connected to Supabase on the Free plan. The documented local interface, application server and AI service used ports 5173, 3000 and 5000. At the time of this report, deployment of the AI service to Microsoft Azure was in progress; completed Azure hosting was not claimed. Table 5.1 records the current development equipment and the confirmed service configuration. These are observed configurations, not measured minimum requirements.

Table 5.1: Implementation environment and evidence

Layer	Configuration	Evidence
Development machine	AMD Ryzen 5 5625U with Radeon Graphics; 7.4 GiB OS-reported usable memory; C: volume 475.8 GiB; Windows 11 Home Single Language, version 10.0.26200	Host inventory on 23 October 2026; current development machine, not a benchmarked minimum
Client browser	Camera-capable browser required; historically tested browser names and versions were not recorded	No browser-compatibility verdict established
Application services	Vue (port 5173), Express (port 3000) and Flask (port 5000) on the development machine	Project README
AI service hosting	Local Flask configuration, port 5000; Microsoft Azure deployment in progress	Project configuration and author confirmation; no completed Azure deployment claimed
Data services	Supabase Free plan: authentication, PostgreSQL and storage; region not recorded	Author confirmation, SQL scripts and supplied database exports
Model experiment	Python 3.13, locked dependencies, saved model version 3.0.0	ML README, requirements lock file, model manifest

5.2.2 Software, Services and Configuration

Vue and Vue Router separated the authentication, verification and administrator screens. Express coordinated authenticated requests and database operations, and Python supplied the image-processing and classification libraries. Table 5.2 lists the dependencies. The package manifests declare version ranges, whereas the model manifest records the exact versions used to produce the saved model.

Table 5.2: Software dependencies and their roles

Component	Version evidence	Role
Vue / Vue Router	^3.5.13 / ^4.5.0	Interface and navigation
Vite	^6.0.7	Development server and frontend build
Express	^4.21.2	Application API and request handling
Supabase JavaScript client	^2.49.1	Authentication and data integration
SimpleWebAuthn	Server ^14.0.3; browser ^14.0.0	Optional passkey authentication
Flask	3.1.3 (model manifest)	AI service endpoints
scikit-learn	1.9.0 (model manifest)	SVM, scaling and calibration (Pedregosa et al., 2011)
NumPy / SciPy	2.5.3 / 1.18.1 (model manifest)	Numerical processing
OpenCV	>=4.8,<5 (requirements file)	Image preprocessing and quality metrics
EasyOCR / InsightFace	Unpinned in requirements file	Text extraction and face embeddings
PyTorch / Silent-Face	PyTorch unpinned; local model weights	Presentation-attack scoring

Backend configuration supplied the Supabase URL and keys, the AI service URL, the listening port and authentication settings. Service-role credentials remained server-side. Optional passkeys required a configured relying-party identifier, allowed origins and Node.js 22 or newer. The isolated classifier environment used Python 3.13.3, scikit-learn 1.9.0, NumPy 2.5.3 and Flask 3.1.3, verified during the 23 October 2026 test run. The full image-processing requirements left EasyOCR, InsightFace and PyTorch unpinned. Those packages were not installed in the inspected general Python environment, so their historically tested versions could not be established. The intended cloud host was Microsoft Azure, with deployment still in progress. An Azure service tier, region and completed deployment endpoint were not established.

5.2.3 Deployment and Reproducibility

The procedure below describes how the repository is installed, configured and verified.

1. Obtain the repository and record the source revision. Create a Supabase project, or use an isolated test project. Apply database/schema.sql to a fresh database, then the access-control, audit, date-of-birth, storage, row-level-security and optional passkey scripts in prerequisite order.
2. Configure the backend and AI service environment files without publishing credentials. Supply the Supabase connection settings, the AI service URL and the frontend origin, and the passkey relying-party settings when passkeys are enabled.
3. Install the backend and frontend packages from their package directories. Create a Python virtual environment and install ai-service/requirements.txt. Keep the scikit-learn version compatible with the supplied model files.
4. Make the InsightFace models and Silent-Face weights available, and keep the complete ai-service/ml/models bundle together with the supporting ml.decision_model module.

5. Start the backend with npm run dev, the AI service with python app.py and the frontend with npm run dev. Check the /health endpoints of the backend and AI service, then call /api/svm/status to confirm that model version 3.0.0 is loaded.
6. Run the isolated tests, then perform a registration with lawful sample data: document capture, selfie capture and verification. Check the request, scores, decision basis and access record in the test database.

The classifier reproduction procedure used ai-service/ml/requirements-experiment.lock.txt and ai-service/ml/improve_model.py with --jobs 3 and --promote. Reproduction requires an isolated copy because promotion replaces the active model bundle, and the runner refuses to repeat a completed experiment on the same evaluation draw. The older train_model.py script produces version 2 artifacts and is not interchangeable with the version 3 workflow. The model was not retrained for this documentation update. Health endpoints provide service-responsiveness checks. Separate live-test evidence was supplied as exports of verification requests, scores and decisions; the observed results are summarised in Table 5.10.

5.3 System Implementation

The implemented architecture separated presentation, application coordination, image analysis and persistence. The browser guided evidence capture, the Express server managed protected operations, and the Flask service produced the numerical evidence used for classification. The subsections below follow this architecture and the Chapter 4 design.

5.3.1 Deviations from the Approved Design

Table 5.3 compares the design choices stated in the project draft with the inspected implementation. The implementation column is supported by code and experiment records; the approved Chapters 3 and 4 were not independently revalidated during this update.

Table 5.3: Deviations from the approved design

Area	Approved design	Implemented	Reason
Training dataset	4,000 records split 70/15/15	10,000 cases in training, calibration, selection and original-test partitions, plus a separate 1,500-case fresh evaluation set	Separate calibration and selection partitions prevented tuning on evaluation data; the original test set was retired once its results had informed model changes
Liveness benchmark	NUAA and Replay-Attack	NUAA only	Only NUAA measurements were recorded; the reason for omitting Replay-Attack was not independently established
Label generation	Labels from literature-based thresholds	Labels from latent scenarios with deliberately overlapping feature distributions	Some uncertainty in real verification is not visible in the six scores, so the classes were not made trivially separable
Model comparison	SVM compared with a Decision Tree	Saved v2 and v3 SVMs and Decision Tree evaluated on the same fresh set; existing threshold rules added as an offline descriptive baseline	A release gate was needed before promoting an improved model

Area	Approved design	Implemented	Reason
Probability output	Class probabilities from the SVM	Sigmoid-calibrated probabilities fitted on a calibration partition	To report more reliable class probabilities
Authenticati on	Email and password, and Google sign-in	Optional passkey (WebAuthn) sign-in added	Faster sign-in on capable devices without storing biometric data
Identity images	Private storage with time-limited signed URLs	Images deleted three business days after the latest successful verification	Data minimisation

5.3.2 Database Implementation

The relational schema represented users, agent profiles and verification attempts as separate entities. UUID primary keys identified records, and foreign keys linked each agent to a user and each request to an agent and document. Unique constraints on user email, the agent-to-user reference and the national identity number limited duplicate accounts and profiles. Verification scores and decisions used unique request references so that each request had at most one score row and one decision row. Table 5.4 summarises the core entities. The implemented schema uses snake_case attribute names. Foreign keys and unique constraints define relationships, but successful processing is needed to create the corresponding score and decision records.

Table 5.4: Core database entities

Entity	Important contents	Purpose
users	user_id, email, role	Authentication-linked account and role
agents	agent_id, user_id, national_id	Profile linked uniquely to a user
documents	document_id, agent_id, file_url	Evidence references for an agent
verification_requests	request_id, document_id, status	Tracks a submitted verification
verification_scores	request_id and six scores	One score record per request
kyc_decisions	decision, probabilities, basis, time	One outcome per request
access_control_records	agent_id, decision_id, access_granted	Records access enforcement
audit_logs	action, outcome, actor, time, details	Traceable application events
passkeys (optional)	credential_id, public_key, counter, label, device and transport metadata	Passkey sign-in

Request status values distinguished pending, verified, review and rejected requests. The decision enumeration excluded pending because it represents completed classifications. The decision_basis field distinguished SVM classifications from placeholder threshold decisions, which allowed stored outputs to be interpreted correctly when model inference was unavailable. Row-level security was enabled in the base schema with policies supplied in separate scripts, and the backend independently validated bearer tokens through Supabase and checked roles before protected actions.

The retention implementation removed stored identity images and selfies three business days after the latest successful verification while preserving decision and audit metadata. Business days were Monday to Friday in the Africa/Nairobi time zone, without public-holiday exclusions. The cleanup job ran at startup and every fifteen minutes, and failed storage removals were retried. The job did not purge pending, review or rejected cases, a limitation discussed in Section 5.7.3.

5.3.3 Application Modules

Authentication, roles and passkeys. Authentication middleware rejected missing or invalid bearer tokens, and role middleware restricted administrator operations. Email/password and Google sign-in were supplemented by WebAuthn passkeys using SimpleWebAuthn. The public.passkeys table stored the credential identifier, public key, signature counter, transports, device type, backup status, label and device metadata; the application did not receive device fingerprint or face templates from WebAuthn. Challenge and rate-limit records were stored separately. Client support detection controlled which passkey actions were offered.

Registration and evidence capture. Agents entered personal details and supplied document and selfie images through the capture interface. Server upload middleware accepted JPEG and PNG MIME types and imposed a 5 MiB limit per file, returning a validation message for unsupported or oversized files. Registration logic checked existing national identity numbers. The storage integration used a private Supabase bucket; its deployed access settings were not live-tested in this update.

Verification pipeline. Four services produced the evidence signals. The quality service computed sharpness as Laplacian variance, brightness as the mean HSV value channel and contrast as the grayscale standard deviation, and images below the configured thresholds were rejected with specific messages before further processing. The OCR service compared the extracted name, identity number and date of birth with the registered details. Face matching compared document-portrait and selfie embeddings generated with the ArcFace implementation in InsightFace (Deng et al., 2019), and liveness scoring used the Silent-Face anti-spoofing models. These were evidence signals: neither text agreement nor facial similarity alone proved that a document was authentic.

The backend sent the evidence to the quality, OCR, face and liveness endpoints and assembled six numerical features, defined in Table 5.5. Raw image-quality measurements were kept in their original units, and MinMax scaling was applied inside the model processing path. Missing, non-finite, wrongly typed or out-of-range evidence was rejected rather than converted to zero.

Table 5.5: Classifier feature contract

Feature	Unit or range	Interpretation
face_match_score	0 to 100	Face similarity score
liveness_score	0 to 1	Anti-spoofing score
ocr_confidence_score	0 to 1	Agreement with registered identity text
blur_score	Non-negative raw value	Laplacian variance; higher means sharper
brightness_score	0 to 255	Mean HSV value-channel intensity
contrast_score	0 to 127.5	Grayscale standard deviation

Decision and access control. The classifier returned Verified, Manual Review or Rejected with class probabilities and model provenance. The backend preserved the lowercase database values verified, review and rejected, and the decision and access-control records connected each classification to the agent's permitted workflow.

Administrator review. The administrator controller implemented approval and rejection, updated the associated decision, request status and access record, and recorded the acting administrator. Dashboard endpoints returned totals by verification status and daily counts. Image viewing used signed URLs, normally valid for up to one hour and further limited by the remaining audit window for verified cases.

The realtime service emitted new_review_case notifications through Socket.IO. These behaviours were confirmed from source code; a live administrator demonstration was not recorded in this update.

Registration-time guardrails. Middleware checked duplicate national identity numbers, blocked disposable email domains, and counts of guardrail events and failed verification initiations from an IP address or device fingerprint. The defaults allowed three counted events in a 24-hour lookback window, with environment variables able to override these values. Duplicate identities produced a conflict response and attempt limits produced a rate-limit response with retry information. These are configured rules, not measured attack-detection rates.

Audit logging. A central logging utility recorded standardised actions and outcomes for registration, each pipeline step, classification decisions, administrator actions, guardrail triggers and passkey events, with the actor and a timestamp.

5.3.4 End-to-End User Workflow

The implemented workflow connected the following interface, application and database operations. The sequence describes code paths; the live execution status is reported separately in Table 5.10.

1. The agent signed up or signed in. The backend validated the token and role, and the guardrail middleware checked the national identity number, the email domain and recent failed attempts.
2. The agent entered personal details, which were stored in the agents table and linked to the user record.
3. The agent uploaded the identity document and captured a selfie. Format and size were validated in the browser and on the server, and the files were stored in the private bucket with a documents record.
4. The backend sent both images to the quality service. A failed check returned specific reasons, such as an image being too blurry or too dark, and asked the agent to resubmit.
5. Images that passed went through OCR, face matching and liveness scoring, and the six scores were stored in verification_scores.
6. The classifier returned a class and probabilities. The backend stored the decision with its decision_basis and wrote the access_control_records entry.
7. The agent saw the outcome. Manual Review cases appeared in the administrator queue, where an approval or rejection updated the decision and access records.
8. Each step wrote an audit_logs entry, so the sequence could be reconstructed for any request.

The supplied live-test evidence consisted of database exports rather than interface screenshots. Their aggregate results are reported in Table 5.10. Personal identifiers and image links are omitted from the chapter. Figures 5.1 and 5.2 show the separate recorded model evaluation.

5.3.5 Exception Handling and Decision Provenance

At the AI endpoint, an unavailable or inconsistent model bundle produced an unavailable-service response. At application level, valid complete evidence could instead enter the existing threshold fallback when SVM inference failed. The fallback recorded placeholder_threshold and an audit reason, and its zero-or-one outputs were rule outcomes and not calibrated confidence. The model-only results in Section 5.4.3 therefore did not measure the combined application, which also includes the quality gates, the fallback and the human-review workflow.

5.3.6 Dataset Description and Preparation

The decision-model dataset contained 10,000 synthetic cases: 6,000 Verified, 2,000 Manual Review and 2,000 Rejected. Each record held the six features in Table 5.5 and a class label. The class quotas were

experimental assumptions and not estimates of operational prevalence. No genuine identity records were used because real agent data raises privacy and consent concerns.

The generator reused stored benchmark measurements: 1,000 LFW face pairs, 1,200 NUAA presentations and 500 MIDV quality frames together with MIDV OCR measurements. The machine learning README and dataset metadata recorded summary statistics and file hashes. These historical measurements informed score distributions; the source images were not reprocessed during the version 3 experiment or this documentation update. None of the benchmark sources supplied the three-class KYC labels.

Face and liveness values were sampled from truncated distributions informed by the recorded measurements. OCR and image-quality values combined measured statistics with explicit simulation assumptions, and quality used a multivariate distribution over transformed blur, brightness and contrast. Some unresolved and genuine scenarios deliberately shared observed distributions to represent information that is absent from the six features. The labels therefore reflected assumed latent scenarios and not a threshold applied to the observed scores.

Exact and near-duplicate checks were run before partitioning. Row identifiers and latent scenario descriptions were excluded from the predictors, and scaling and optional quality transformations were fitted within training folds to avoid leakage. These checks screened close observations but could not show that the simulated population represents real applicants. The licence and redistribution terms of the benchmark datasets were not assessed here, and no redistribution claim is made.

Table 5.6: Data partitions used in the version 3 experiment

Partition	Cases	Function
Training	7,000	Model fitting and five-fold cross-validation
Calibration	750	Probability calibration only
Selection	750	Candidate and decision-policy selection
Original test	1,500	Retired from version 3 selection and fitting
Fresh evaluation	1,500	Frozen comparison of the candidate and the incumbent

The calibration and selection partitions each contained 450 Verified, 150 Manual Review and 150 Rejected cases, and the fresh evaluation set contained 900, 300 and 300 respectively. The fresh set was drawn after candidate selection was frozen, and duplicate checks covered all 11,500 original and fresh records. The original test set was retired because earlier results on it had informed the new research questions.

5.3.7 Model Training, Selection and Packaging

An RBF support vector machine combined the six features into a three-class decision (Pedregosa et al., 2011). Training compared C values of 0.1, 1, 10 and 100 with gamma values of 0.001, 0.01, 0.1, 1, scale and auto, giving 24 combinations per grid. Five-fold shuffled stratified cross-validation used macro F1 as the score, balanced class weights and seed 42, and preprocessing stayed inside the cross-validation pipeline.

For each parameter combination, the 7,000 training cases were divided into five stratified folds of approximately 1,400 cases. A fresh SVM was fitted on four folds (approximately 5,600 cases) and assessed on the remaining fold; the held-out fold changed across five rounds. Macro F1 was averaged

across the five validation results. The 24 combinations therefore required 120 fold-level fits per grid, followed by refitting the selected configuration on all 7,000 training cases. Scaling was fitted only on the training portion of each fold. The calibration, selection and fresh evaluation cases were excluded from this cross-validation process.

The version 3 experiment compared raw and transformed quality inputs, native multiclass and explicit one-versus-rest SVM variants, and sigmoid, isotonic and temperature calibration. Three additional grids were fitted, and the earlier raw-native winner was reused with its provenance recorded. Calibration used only the calibration partition. Candidate selection used the separate selection partition and applied safeguards on macro F1, review recall and both false-verification counts relative to the comparable calibration baseline.

The selected configuration was raw_native_sigmoid with C = 10 and gamma = scale, and its training cross-validation macro F1 was 0.672766. Its probability policy used a review multiplier of 1.0, a verification threshold of 0.0 and a verification margin of 0.0, so no extra conservative verification threshold was applied beyond the selected policy.

The release gate required strictly higher accuracy on the fresh set, non-decreasing macro F1, and no increase in either Rejected-to-Verified or Manual-Review-to-Verified errors relative to the saved version 2 incumbent on the same fresh records. The selected candidate passed the gate and was promoted to the local version 3.0.0 bundle. Passing the gate met the specified comparison; it did not establish an acceptable operational false-approval rate.

The model manifest recorded the feature order, class names, library versions and artifact hashes. The service validated the bundle at startup and mapped probabilities using the model's class labels. The saved model, scaler and supporting code are required together. The model stayed static during operation, so completed verifications did not trigger online learning, and legacy artifacts were kept separately to allow comparison and rollback.

Table 5.7: Model version history

Version	Main change	Result on its evaluation set
Version 1	4,000-row synthetic generator with largely separable class ranges; earlier scaler/CV methodology	100.000% accuracy and macro F1 1.000000 on 600 historical test cases; artificially easy separation limits interpretation
Version 2	10,000 scenarios with overlap, fold-local scaling, training-only tuning and held-out calibration	74.333% accuracy and macro F1 0.621383 on its original 1,500-case test set
Version 3.0.0	Calibrated raw-input SVM, C = 10, gamma = scale	75.800% accuracy, macro F1 0.654082 on the fresh set

Results for versions 1 and 2 came from earlier evaluation sets and are not directly comparable with version 3. The like-for-like comparison of version 2 and version 3 on the same fresh records is in Table 5.11.

5.4 Testing and Evaluation

5.4.1 Test Strategy and Acceptance Criteria

Automated testing covered classifier contracts, experiment integrity, backend decisions, retention and passkey controls. The three Python suites used unittest from Python 3.13.3; backend suites used Node.js 24.19.0 and its built-in node:test runner. Tests used synthetic features, test credentials, mocks and an embedded PostgreSQL database for the passkey migration. The recorded version 3 log contained 14

passing tests. On 23 October 2026, those tests were rerun and the scope was expanded as summarised in Table 5.9. Three Supabase exports additionally documented live request, score and decision records. These observations were assessed separately from automated assertions and did not establish participant-based usability. Requirement identifiers in Table 5.8 are retained from the supplied draft.

The classifier was evaluated with accuracy, macro and weighted F1, class-specific precision and recall, log loss, the multiclass Brier score and the counts of non-Verified cases predicted as Verified, because those errors give access to agents who should not receive it.

5.4.2 Functional Test Cases and Results

Table 5.8 summarises representative automated test cases. All nine scenarios were covered by the saved version 3 test log and the successful rerun on 23 October 2026. Each pass required the executable assertions to agree with the expected outcome, rather than a manual judgement that the interface appeared to work.

Table 5.8: Isolated test cases and results

ID	Traceability	Scenario and expected result	Result
TC01	FR-13, NFR-10	Missing, malformed, non-finite and out-of-range values are rejected	Pass
TC02	FR-13	With the class order varied, each probability maps to its class label	Pass
TC03	FR-13, NFR-13	Saved artifact and API predictions agree, and legacy API fields are retained	Pass
TC04	NFR-09	With the model unavailable, no AI-service prediction is invented	Pass
TC05	Data requirements (4.2.3)	Regenerated data are reproducible and hidden predictors are excluded	Pass
TC06	Data requirements (4.2.3)	Split manifest and scaler show preprocessing fitted on training data only	Pass
TC07	FR-13, FR-15	The review policy is applied and original probabilities are preserved	Pass
TC08	FR-12, FR-13, NFR-10	Incomplete scores fail before inference or threshold fallback	Pass
TC09	FR-14 to FR-16, FR-24, NFR-13	A model prediction preserves database enums and the decision basis	Pass

Table 5.9: Recorded suite totals

Suite	Tests	Result
test_svm.py	6	6 passed
test_kyc_experiment.py	2	2 passed
test_kyc_improvement.py	4	4 passed
kycDecision.test.js	2	2 passed
documentRetention.test.js	5	5 passed
passkeys.test.js	9	9 passed
passkeyRoutes.test.js	1	1 passed
passkeyDatabase.test.js	1	1 passed
Frontend unit tests (Vitest 5.0.3)	16	16 passed

Suite	Tests	Result
Expanded run total	46	46 passed; no assertion failures in completed suites

The expanded run on 23 October 2026 passed 46 automated tests, as shown in Table 5.9. Python used unittest in .venv-kyc; backend tests used node:test, with PGlite for the passkey migration and mocks for storage and session boundaries. Frontend tests used Vitest with jsdom. The original 14 tests are included in this total, not additional to it. Logs are retained in documentation/final_draft_evidence. Table 5.10 separately summarises the live Supabase exports and the limits of the available system-level evidence.

Frontend startup failed initially because of an invocation-path error and then a filesystem restriction. After correcting the execution setup without changing application code, all 16 tests passed. Initial outputs and the successful rerun were retained.

The supplied files were Supabase Snippet Untitled query.csv (decisions), query (1).csv (requests) and query (2).csv (scores). Their timestamps covered 21 September to 3 October 2026 in Africa/Nairobi. Nine decision probability triples summed to one, and every exported decision linked to a request with matching status. These records show stored outputs, not classification accuracy: they contain no independently adjudicated ground truth, no model-version field and no access-control or audit-log export. Historical threshold outputs must not be assumed to replay under the current rule revision.

Table 5.10: Live test observations and system-level coverage

Area	Evidence	Observed result	Interpretation	Status
Request and score storage	Request and score CSV exports	12 requests across 8 agent identifiers; all 12 linked to six-score records	Supports persistence of verification evidence	Observed
Decision output	Decision CSV joined by request_id	9 linked decisions: 7 Verified, 2 Rejected; all matched request status	Supports recorded outcome generation; no Manual Review example	Observed
Decision provenance	decision_basis values	5 svm_model and 4 placeholder_threshold decisions	Both paths occurred; model version and fallback cause were not recorded	Observed
Incomplete attempts	Request-to-decision join	3 pending requests had no decision in the supplied exports	Completion was not demonstrated for every recorded attempt	Open
Passkeys and retention	Isolated automated test logs	Passkey service, route and migration checks and 5 retention tests passed	Does not certify physical authenticators or live storage cleanup	Locally tested
Other system requirements	No corresponding records supplied	Access grants, administrator actions, negative security cases, latency and participant usability not established	Requires separate records; no pass/fail result assigned	Not verified

5.4.3 Model Evaluation Results

The saved version 2 incumbent, selected version 3 SVM and saved Decision Tree reference were evaluated on the same 1,500 fresh synthetic cases in the recorded version 3 experiment. The Decision Tree values were recovered from the reference_models section of improvement_report.json; the tree was not retrained or selected using this evaluation set. On 23 October 2026, the unchanged applyPlaceholderDecision function was additionally executed offline on those same cases. This post-hoc threshold comparison is descriptive and did not influence the earlier SVM selection or release gate. Table 5.11 reports all four approaches.

Table 5.11: Model comparison on the same fresh evaluation set

Metric	Saved v2	Selected v3	Decision Tree	Threshold rules
Accuracy	75.733%	75.800%	75.667%	28.667%
Macro F1	0.651909	0.654082	0.648469	0.263647
Weighted F1	0.730391	0.731979	0.726834	0.212398
Rejected predicted Verified	44	43	95	0
Manual Review predicted Verified	186	184	208	8
Log loss	0.660123	0.659596	0.789836	Not applicable
Multiclass Brier score	0.373659	0.373110	0.461111	Not applicable

The accuracy difference between version 3 and version 2 was 0.067 percentage points, equivalent to one additional correct decision. The paired-bootstrap 95% interval for that difference ran from −0.133 to +0.333 percentage points. The macro F1 difference was 0.002173, with an interval from −0.001051 to +0.006855. Both intervals include zero, so the results do not show a conclusive improvement. They describe uncertainty for the fitted models under the same simulator, not across real populations or all possible training runs.

Table 5.12: Selected version 3 confusion matrix

Actual class	Predicted Verified	Predicted Review	Predicted Rejected
Verified	831	15	54
Manual Review	184	82	34
Rejected	43	33	224

Table 5.13: Selected version 3 class-specific performance

Class	Precision	Recall	F1
Verified	0.785444	0.923333	0.848825
Manual Review	0.630769	0.273333	0.381395
Rejected	0.717949	0.746667	0.732026

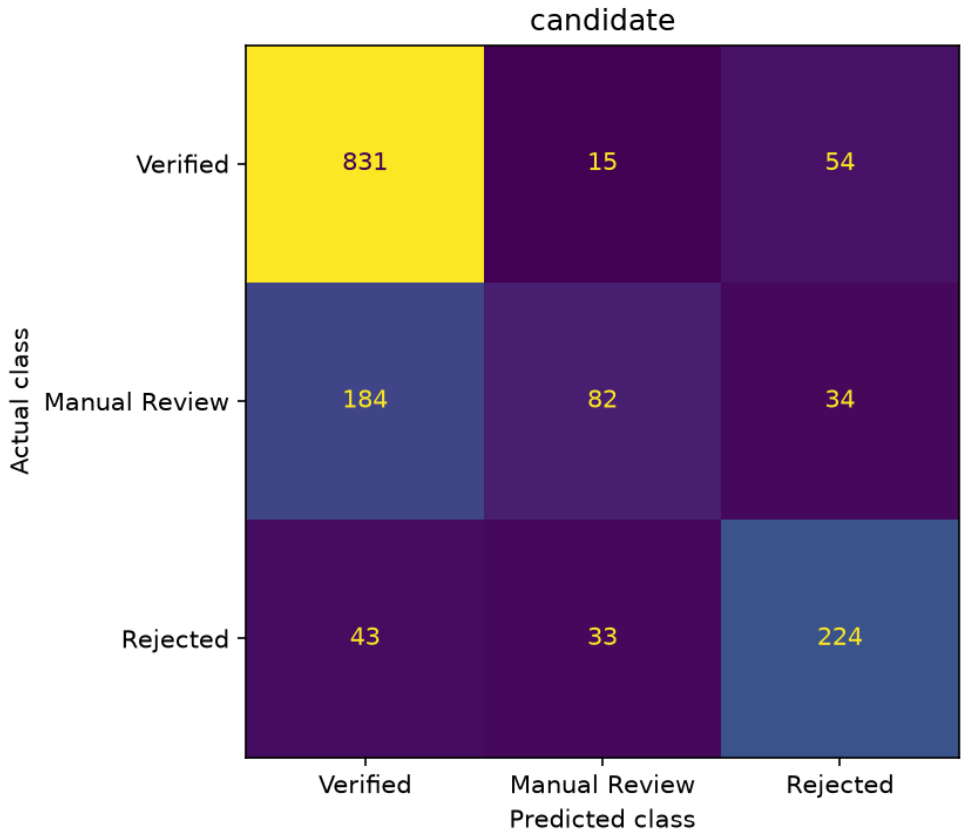


Figure 5.1: Version 3 confusion matrix on 1,500 fresh synthetic cases. Source: saved version 3 evaluation artifacts.

Manual Review was the weakest class: only 82 of 300 cases were retained for review, while 184 were verified. A further 43 of 300 rejected cases were verified. Together these gave 227 false verifications among 600 non-Verified cases, or 37.83%. Of the 1,058 predicted verifications, 227 were incorrect under the synthetic labels, or 21.46%. These rates have different denominators and should not be confused with the 24.20% overall classification error.

The Decision Tree achieved 75.667% accuracy and macro F1 of 0.648469, compared with 75.800% and 0.654082 for version 3. The SVM therefore exceeded the tree on these point estimates, but no paired significance test between these two models was recorded. The threshold rules achieved 28.667% accuracy and macro F1 of 0.263647. They verified no Rejected cases and only eight Manual Review cases, but rejected 471 of the 900 Verified cases and sent another 363 to review. Their low false-verification count therefore came with substantially poorer acceptance of genuine simulated cases. Rule outputs were not calibrated probabilities, so log loss and Brier scores were not reported for that baseline.

5.4.4 Error Analysis and Interpretation

Class imbalance and overlapping scenarios explain why accuracy alone overstated useful review performance. Verified cases made up 60% of the fresh sample and had high recall, whereas Manual Review cases frequently resembled Verified cases in the observed feature space. The simulator deliberately included indistinguishable components with different latent labels, and the stored identifiability analysis attributed a minimum eight-percentage-point population error contribution to them,

with further overlap adding more. This is a property of the simulation and not an estimated limit for real identity verification.

Table 5.14: Permutation importance for the selected model

Feature	Macro F1 decrease	Repeat standard deviation
Liveness	0.201380	0.008477
Face match	0.110442	0.005995
OCR agreement	0.049802	0.007588
Blur	0.003831	0.003357
Brightness	−0.000359	0.003924
Contrast	−0.003439	0.003605

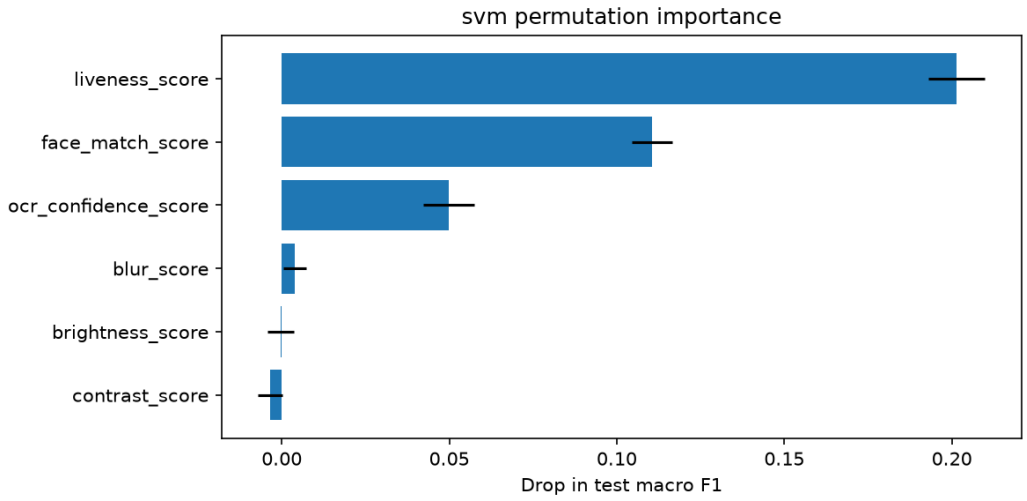


Figure 5.2: Version 3 permutation importance measured by the decrease in macro F1. Error bars show variation across repeated shuffles. Source: saved version 3 evaluation artifacts.

Liveness and face matching had the strongest measured influence on classification, and OCR added further information. Brightness and contrast showed negligible or slightly negative average importance. A negative value means that shuffling the feature slightly improved the measured score in that experiment; it does not show that image quality is useless, because quality also affected capture and text extraction before the classifier, and correlated features can mask one another.

Log loss and Brier score improved slightly, but probability calibration remains conditional on the simulated 60/20/20 class mixture. A score read as confidence under that mixture is not automatically a trustworthy probability for a different applicant population. Errors that grant access to a non-Verified case also have a different operational consequence from errors that send a genuine applicant to review, which is why the two false-verification counts were reported alongside the aggregate metrics.

5.5 Version Control and Implementation Evidence

Project repository: https://github.com/kar-iuki/Intelligent-Agent-Identity-Verification-System

The repository contained the frontend, backend, AI service, database scripts, dependency files, tests and saved experiment reports. The inspected local HEAD was a8f64cb, dated 20 October 2026. Its reachable history contained 25 commits dated from 1 September to 20 October 2026. No release tags were present.

The revised chapter and its new evidence files were local additions and were not represented by that commit. Legacy model bundles were retained for comparison and rollback. Supervisor access and a repository-wide licence were not verified; no public reuse permission is asserted here.

The .gitignore excluded .env and .env.local files, dependency folders, build output, local AI caches and selected test artifacts. These exclusions do not prove that credentials were never committed or that all private datasets are excluded. A complete repository-history secret audit was not performed, and no claim of a clean historical secret scan or credential rotation is made.

5.6 Author Oversight and Use of AI-Assisted Tools

Claude (Anthropic) was identified in the supplied draft as assisting with documentation and implementation prompts. OpenAI Codex assisted with reviewing the source and saved reports, preparing this chapter, executing the isolated automated tests and the offline threshold comparison, and checking the Word layout. Its changes were checked against the inspected code and retained test outputs. The student remains responsible for reviewing the final chapter, verifying alignment with the approved objectives and references, and approving the submission.

5.7 Evaluation Results, Discussion and Limitations

5.7.1 Assessment Against the Objectives

Table 5.15 relates the evidence in this chapter to the five project objectives.

Table 5.15: Objectives, evidence and results

Objective	Output and evidence	Result
1. Investigate current registration and verification practice	Literature review, Chapter 2	Addressed in Chapter 2 and used to derive the Chapter 4 requirements
2. Identify challenges and fraud risks	Chapter 2; countermeasures implemented as quality gates, liveness scoring and guardrails	Addressed; countermeasures implemented
3. Examine existing KYC platforms	Related-works comparison and research gap, Chapter 2	Addressed in Chapter 2
4. Develop the system	Modules in Section 5.3; version 3.0.0 classifier	Implemented; classifier reached 75.800% accuracy and 0.654082 macro F1 on the fresh synthetic set
5. Test and validate effectiveness	Tables 5.8 and 5.9: automated test evidence; Table 5.10: live evaluation coverage; Tables 5.11–5.14: classifier and baseline results	Partly met: 46 automated tests passed; live exports showed 12 score-linked requests and 9 decisions. Real-world accuracy, complete security coverage and usability remain unestablished.

5.7.2 Discussion

The implementation combined evidence capture, numerical scoring and persistent verification decisions. The live exports supported this path through 12 request/score pairs and nine matching decisions. The automated tests added evidence for selected interfaces, input handling, passkeys and retention. However, three attempts remained pending, and access enforcement, administrator intervention and user acceptance were not established by the supplied exports.

For classification, version 3 achieved 75.800% accuracy and 0.654082 macro F1 on the fresh synthetic sample and passed its comparative release gate. The improvement over version 2 was inconclusive. Liveness and face matching carried most of the measured importance, which is consistent with the emphasis in the literature on liveness detection (Tan et al., 2014) and discriminative face embeddings (Deng et al., 2019) and supports the pipeline design in Chapter 3. The low Manual Review recall of 27.33% and the 227 false verifications show that the model should not approve agents autonomously. The Manual Review route and the administrator override exist to catch uncertain cases, but the experiment did not measure how often administrators would correct these errors.

Decision provenance distinguished SVM outputs from threshold-rule outputs in the live records. Token validation, role checks, guardrails and image retention were implemented, but the exports did not include access-control or audit records that would prove their live enforcement. The successful isolated passkey and retention tests supplied narrower evidence for those modules; no general security or privacy-compliance conclusion was drawn.

5.7.3 Limitations

Data validity was the main limitation. The labels were synthetic, the class quotas were fixed and the score relationships were assumed. The fresh evaluation sample came from the same simulator and not from independent real applicants, and the benchmark measurements were reused from earlier runs, so later changes to the image pipeline could cause a mismatch between the source distributions and deployed scores.

Evaluation coverage remained limited. The live evidence comprised 12 requests from eight agent identifiers, with no independent identity labels or recorded model versions; it could not establish real-world accuracy. No demographic fairness study, presentation-attack campaign, camera comparison, participant usability study or timed ten-run pipeline benchmark was documented. Small calibration and selection partitions also left a risk of policy overfitting. Model metrics excluded the full effects of capture quality gates, fallback and administrator intervention.

The retention job removed images only for verified agents, while pending, review and rejected cases were excluded from the automated purge. The local retention tests confirmed that exclusion, so it remained a data-minimisation limitation. Unpinned image-processing dependencies and the absence of a historical environment snapshot also limited exact reproduction of the complete pipeline.

The evidence supports a functioning implementation and a reproducible synthetic classifier experiment. The real-world effectiveness of the classifier remains unestablished. Conclusions, recommendations and future work, including periodic retraining on administrator-confirmed decisions, are presented in Chapter 6.